



暨南大學
JINAN UNIVERSITY

Undergraduate Lab Report

Course Title: Experiment of Computer Organization

Course No: 60080014

Student Name: 李榮耀

Student No: 2024103975

School: International School

Department: _____

Major: CST

Instructor: SUN Heng

Academic Year: 2025~2026, Semester : 1st [☒] 2nd [☐]

Academic Affairs Office of Jinan University

Date (dd/mm/yyyy) 2025/11/19

Computer Organization Lab List

Student Name: Li Rongyao Student No:

2024103975

ID	Lab Name	Type
1	Number Storage Lab	Individual
2	Manipulating Bits	Individual
3	A Simple Real-life Control System Lab	Team
4	System I/O	Individual
5	Simulating Y86-64 Program	Individual
6	Performance Lab	Team

Course Title _____ Experiment _____ of _____ computer _____ organization _____

Evaluation _____

Lab Name _____ System I/O _____

Instructor _____ Sun Heng _____

Lab Address _____ Room 116 _____

Student Name _____ Li Rongyao _____

Student No _____ 2024103975 _____

College _____ International School _____

Department _____ Major _____ CST _____

Date 2025/11/19 _____ Afternoon _____

The purpose of this lab is to become more familiar with the Unix-I/O by modifying the file to handle different problems of Unix-I/O.

The C programs are provided. These codes are available from the lab materials. Related concepts can be found in the file *04.ppt* of our theoretical course.

a) Modify the *cpfile* program so that it uses the Rio unbuffered functions *Rio_readn* and *Rio_writen* to copy standard input to standard output, 10 bytes at a time. Use **\$ make** to compile, and then use **\$./main.out** to run. Remember to use **& make** again after modifying *cpfile.c*;

- b) Modify the original *cpfile* program so that it takes an optional

command line argument *infile*, such as: `$ ./main.out infile.txt`; If *infile* is given, then copy *infile* to standard output; otherwise, copy standard input (`STDIN_FILENO`) to standard output as before. The twist is that your program must use the original copy loop (lines 13–15) for both cases. You are only allowed to insert code, and you are not allowed to change any of the existing code.

3. Lab Device or Environment

Ubuntu 16.04 (64-bit) on virtual machine (VMware workstation 17 Pro)

4. Results and Analysis

Activity a):

Results:

```
amikan-honor@amikanhonor-virtual-machine:~/文档/lab4$ make
gcc -c -o cpfile.o cpfile.c
gcc -c -o csapp.o csapp.c
gcc cpfile.c csapp.c -lpthread -o main.out
amikan-honor@amikanhonor-virtual-machine:~/文档/lab4$ ./main.out
helloworld
helloworld
What can I say?

What can
```

Analysis:

First, we know the prototype of *Rio_readn* and *Rio_writen* are:

```
ssize_t Rio_readn(int fd, void *usrbuf, size_t n)
Wrappers for Rio package
Rio_readn

void Rio_writen(int fd, void *usrbuf, size_t n)
Rio_writen
```

fd is the descriptor of file, *usrbuf* is the pointer at the buffer of users, and *n* is the size of read or write. It performs the ability to read data directly

from file without buffer.

So, to modify *cpfile*, we need to modify the read function as *Rio_readn* and let *size* equals to 10 as the argument of the function.

In my example, the first line have 11 bytes “*helloworld\n*”, then the output is “*helloworld*” as 10 bytes at a time. Then my input is “*\nWhat can I say?\n*”, the program continue to output the rest byte in the standard input and then read more bytes until 10 bytes. Thus the second output is “*\n\nWhat can*”. That verifies the correctness of my program.

Activity b):

```
amikan-honor@amikanhonor-virtual-machine:~/文档/lab4$ make
gcc -c -o cpfile.o cpfile.c
gcc -c -o csapp.o csapp.c
gcc cpfile.c csapp.c -lpthread -o main.out
amikan-honor@amikanhonor-virtual-machine:~/文档/lab4$ ./main.out
Hello, world!
Hello, world!
Man! What can I say? Mamba out!
Man! What can I say? Mamba out!
amikan-honor@amikanhonor-virtual-machine:~/文档/lab4$ ./main.out infile.txt
I/O Redirection is successful!amikan-honor@amikanhonor-virtual-machine:~/文档/lab4$
```

Analysis:

We need to handle the situation that the command line containing the *infile.txt*, which is the second element of *argv*. So we need a condition judgment to handle this situation.

First, we need to open the *infile.txt* by *open* and get a file descriptor. Then handle the error situation. The core operation is to use *dup2* to let the *STDIN_FILENO* points at the same destination of *fd*, which points at *infile.txt*, then we can use the same loop to perform that copy the *infile.txt* to standard output. Remember to use *close* free the file descriptor of *fd*.

We can also re-initialize the buffer *rio* by *Rio_readinitb* to the *infile.txt*. But it not matches the idea of redirection. So we use *dup2* rather than re-initialize.

In my example, first just test *main.out* in command line. The output is the same as before from the standard input. Then takes the input *infile.txt*, the output is correct.

5. Appendix (Program Code)

cpfile_1.c (for activity a)

```
/* All of rio function implementations are obtained from csapp.c */
/* Please note the usage of wrapper function */

/* $begin cpfile */
#include "csapp.h"

int main(int argc, char **argv)
{
    int n;
    int size = 10;
    char buf[MAXLINE];

    while((n = Rio_readn(STDIN_FILENO, buf, size)) != 0)
        Rio_writen(STDOUT_FILENO, buf, n);
    exit(0);
}
/* $end cpfile */
```

cpfile_2.c (for activity b)

```
/* All of rio function implementations are obtained from csapp.c */
/* Please note the usage of wrapper function */

/* $begin cpfile */
#include "csapp.h"

int main(int argc, char **argv)
```

```
{
    int n;
    rio_t rio;
    char buf[MAXLINE];

    Rio_readinitb(&rio, STDIN_FILENO);

    if (argc == 2)
    {
        int fd = open(argv[1], O_RDONLY);
        if (fd == -1){
            exit(0);
        }
        Dup2(fd, STDIN_FILENO);
        close(fd);
    }

    while ((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0)
        Rio_writen(STDOUT_FILENO, buf, n);

    exit(0);
}
/* $end cpfile */
```